

Exercise 4 Scheduling Methods and Process Switching

Learning Objectives:

- Estimate the runtimes of various scheduling methods
- Practical influence of process scheduling on Linux

Legend:

 Reading that must be completed before the exercise

 Questions/tasks to be completed before the exercise

❖ Tasks to be completed during the exercise

4.1 Preparation for the Exercise

The preparations and questions should be completed before the exercise session. The exception is this first exercise, for which you have not yet had an opportunity to read the material. They are also intended as aids to help you with the tasks to be completed during the exercise sessions themselves.

 To prepare, study the following materials

- Chapters 8.6.1 ff. + 8.6.12 of Chr. Baun, Operating Systems, 2nd Edition,

 **Introductory Questions:**

When is a scheduling method considered *fair*?

Which scheduling methods are fair?

- ☐ Priority-based scheduling
- ☐ First Come First Served
- ☐ Round Robin

- ☐ Earliest Deadline First
- ☐ Fair-Share

What does preemptive mean in scheduling methods?

Which scheduling methods are preemptive?

- ☐ First Come First Served
- ☐ Round Robin
- ☐ Earliest Deadline First
- ☐ Fair-Share



Research Required:

Unix systems use priority-based scheduling in part, where the priorities of user processes can also be changed at runtime. Research which priority ranges exist under Linux and which of these are available to user processes.

Also find out how the priority of a user process can be set by the user at program start as well as how the priority can also be changed dynamically.

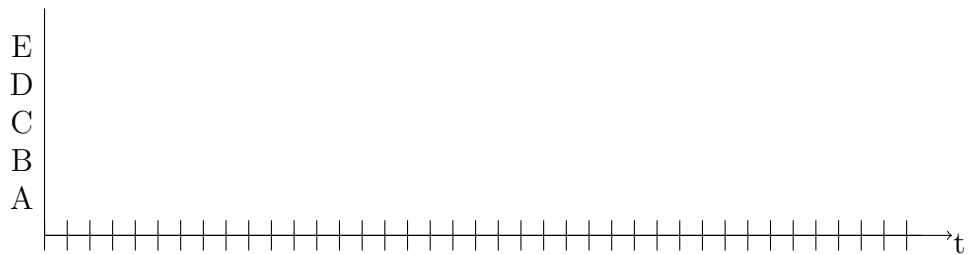
4.2 Scheduling Procedures

❖ On a processor with a single core, five processes are being executed. All processes are in the *ready* state at time $t_0 = 0$. The priority increases with the priority number, and blocking is not considered in this scenario.

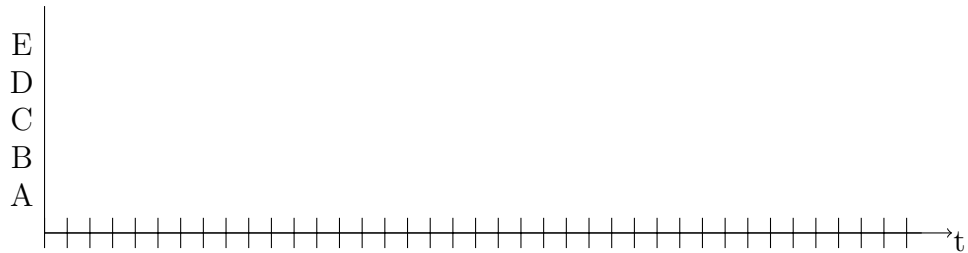
Process	Execution Time	Priority
A	10 ms	15
B	5 ms	5
C	6 ms	4
D	3 ms	12
E	8 ms	7

Draw the execution order of the processes in a time diagram for:

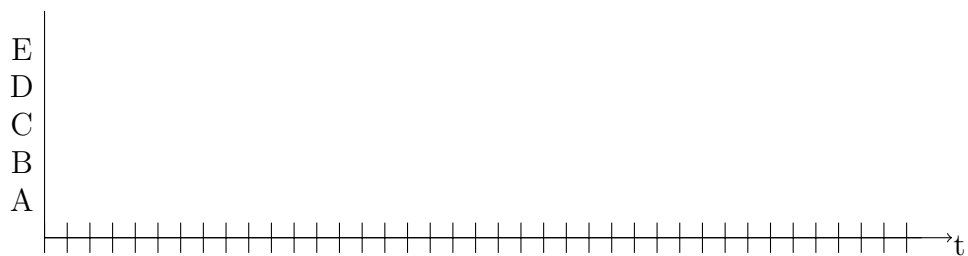
- a) Round Robin with a time quantum $q = 2ms$



- b) First Come First Served with processes arriving in the order A, B, C, D, E



- c) Priority-based scheduling with static priorities



- d) Calculate the total run time for each process for all the scheduling methods above. The run time is the duration between the process entering the *ready* state and its completion.

- e) Also calculate the average run time $\bar{\theta}$ for each method.

Process	Execution Time	Run Time		
		RR	FCFS	Priority Sched
A	10 ms			
B	5 ms			
C	6 ms			
D	3 ms			
E	8 ms			
∅	—			

- f) Calculate the waiting times for each process for all the scheduling methods above. Waiting times refer to the time each process spends in the *ready* state.
- g) Also calculate the average waiting time $\emptyset$ for each method.

Process	Execution Time	Waiting Time		
		RR	FCFS	Priority Sched
A	10 ms			
B	5 ms			
C	6 ms			
D	3 ms			
E	8 ms			
∅	—			

4.3 Adjusting Process Priority

 As preparation, you should familiarize yourself with the functionality of the C program in Listing 1.

❖ The following C program measures and outputs its own execution time. The nested loops generate a sufficiently long execution time.

- Compile the C program.
- Run the program at least five times with the default priority, and record the execution time each time. If you compiled it with Eclipse IDE, you find the program under `<home_dir>/<workspace_dir>/<projectname>/Debug/`
- Now run the program at a higher priority, again at least five times, and record the execution time each time.
- What do you observe when comparing the times?

Program Run	Execution Time with Process Priority =	Execution Time with Process Priority =
1		
2		
3		
4		
5		
∅		

Listing 1: proc_execution_times.c

```

#include <stdio.h>
#include <errno.h>
#include <sys/times.h>
#include <unistd.h>
#include <time.h>

#define MAX 100000

int main()
{
    struct tms exec_time;
    clock_t act_time;

    long ticks_per_second;
    long tickduration_in_ms;

    unsigned long i = 0;
    unsigned long counter = 0;

    ticks_per_second = sysconf(_SC_CLK_TCK);
    tickduration_in_ms = 1000/ticks_per_second;

    for (i = 0; i < MAX; i++)

```

```
{
    for (counter = 0; counter < i; counter = counter + 1)
    {
        if (!(counter%10000))
        {
            printf("%ld:\n", counter);
        }
    }
}

act_time = times( &exec_time );
if (act_time >= 0)
{
    printf("\n");
    printf("execution time (in ms): %ld\n",
           (exec_time.tms_utime+exec_time.tms_stime)*tickduration_in_ms);
}
else
{
    fprintf(stderr, "Error: %d",errno);
}
return 0;
}
```